

07 로지스틱 회귀 노트북 작업 정리

대상 파일: 안성준/01 Santander customer satisfaction/07 로지스틱 회귀.ipynb | 작성일: 2026-08-20

1. 작업 개요

Santander Customer Satisfaction(캐글) 데이터로 학습한 로지스틱 회귀 베이스라인 노트북에 두 가지 개선 작업을 순서대로 추가했습니다. 두 작업 모두 기존 베이스라인 셀은 그대로 두고, 새 변수명을 사용한 별도 섹션으로 추가하여 이전 결과와 비교할 수 있게 구성했습니다.

작업	목적	핵심 기법
① SMOTE 오버샘플링	클래스 불균형(96:4) 완화	imblearn.SMOTE로 학습 데이터만 합성 오버샘플링
② 참고 자료 기반 전처리	불필요/중복 피처 정리 + 피처 엔지니어링	중복.희소 컬럼 제거, var15/var38 피처 엔지니어링

2. SMOTE 오버샘플링 적용

TARGET 값이 0(만족):1(불만족) = 96:4로 심하게 불균형해, 모든 샘플을 다수 클래스로만 예측해도 accuracy가 높게 나오는 문제가 있었습니다. 이를 보완하기 위해 소수 클래스를 합성 오버샘플링하는 SMOTE(Synthetic Minority Oversampling Technique)를 적용했습니다.

2.1 적용 방법

- 반드시 train_test_split 이후, 학습 데이터(X_train, y_train)에만 SMOTE를 적용 (데이터 누수 방지)
- 테스트 데이터(X_test, y_test)는 원본 분포 그대로 유지해 실제 배포 환경과 동일한 조건에서 평가
- 소수 클래스(TARGET=1)를 다수 클래스와 1:1 비율로 합성 (2,401건 → 58,415건)
- 리샘플링된 데이터로 LogisticRegression을 재학습하고 원본 X_test로 평가

2.2 교차검증 시 주의사항

GridSearchCV/cross_val_score처럼 교차검증을 함께 쓸 경우, 학습 데이터 전체에 SMOTE를 미리 적용하면 검증 폴드에도 합성 샘플 정보가 섞여 성능이 과대평가(leakage)됩니다. 이 경우 sklearn.pipeline.Pipeline 대신 imblearn.pipeline.Pipeline을 사용해 SMOTE를 파이프라인 단계로 넣으면 각 CV 폴드의 학습 폴드에만 SMOTE가 적용되어 누수를 막을 수 있습니다.

2.3 결과

구분	accuracy	roc_auc	클래스1 recall
SMOTE 적용 전	0.959	0.786	-
SMOTE 적용 후	0.705	0.791	0.72

accuracy는 다수 클래스 몰빵 예측이 사라지며 크게 하락하지만, roc_auc는 소폭 개선되고 불만족 고객(클래스 1)에 대한 recall이 0.72까지 상승합니다. 불균형 데이터에서는 accuracy가 아닌 roc_auc-recall을 기준으로 판단해야 함을 보여주는 결과입니다.

3. 참고 자료 기반 추가 전처리

아래 글과 글이 인용하는 GitHub 구현(EDA.ipynb)의 실제 전처리 코드를 참고해, 이 데이터셋에 맞게 검증 후 반영했습니다.

· 글: Santander Customer Satisfaction - A self case study using Python (towardsdatascience.com)

· 코드 출처: github.com/ashishthomaschempolil/Santander-Customer-Satisfaction (EDA.ipynb)

3.1 적용 단계

- 중복 컬럼 제거 — 값이 완전히 동일한 컬럼 쌍을 찾아 하나만 유지 (29개 발견)
- 희소(sparse) 피처 제거 — 상위 99% 값이 0인(거의 대부분 0인) 컬럼 제거 (165개 발견)
- var15 피처 엔지니어링 — 23세 미만 여부를 나타내는 var15_below_23 이진 피처 추가, 연속값을 5구간으로 binning
- var38 로그 변환 — 대출금액(var38)의 우편향(right-skew) 분포 완화를 위해 np.log 적용

참고로, 기존 노트북에 이미 있던 분산 0(zero-variance) 컬럼 탐지 코드는 계산만 하고 실제로는 사용되지 않고 있었는데, 이번 작업에서 이를 실제로 체이닝해 최종 피쳐셋에 반영했습니다.

3.2 결과

구분	피처 수	accuracy	roc_auc
베이스라인 (원본)	369	0.959	0.786
SMOTE 적용	369	0.705	0.791
참고 자료 전처리 적용	142	0.960	0.812

369개 원본 피처가 중복/희소 컬럼 제거만으로 142개로 줄었으며, 이는 참고 글에서 보고한 "정제 후 142개 피처"와 거의 일치합니다(개별 단계별 수치는 글의 서술과 다소 달랐으나, 이 train.csv에 동일 규칙을 직접 적용해 얻은 실측치를 기준으로 삼았습니다). var15/var38 피처 엔지니어링까지 반영한 결과 roc_auc가 0.786 → 0.812로 개선되었고 accuracy도 유지되어, 이 데이터셋에서는 단순 SMOTE보다 피처 정제 자체가 더 효과적이었습니다.

4. 종합 비교

버전	피처 수	accuracy	roc_auc	비고
베이스라인	369	0.959	0.786	분산 0 컬럼 미적용 상태
SMOTE	369	0.705	0.791	recall 대폭 개선, accuracy 하락
참고 자료 전처리	142	0.960	0.812	현재까지 roc_auc 최고

전체 노트북 구조: 데이터 로드 → 불균형 확인 → (기존) 분산 0 컬럼 확인 → 표준화/학습 → 베이스라인 결과 → SMOTE 섹션 → 참고 자료 기반 전처리 섹션 → solver 비교 → GridSearchCV → (관련 없는 보스턴 주택가격 예제 잔존).

본 문서는 07 로지스틱 회귀.ipynb에 추가된 작업 내용을 요약한 정리 자료입니다.